Formal Security Analysis of the MACsec Key Agreement (MKA) Protocol using Tamarin

[1]Halil İbrahim Kaplan

Tübitak BİLGEMhalil.kaplan@tubitak.gov.tr

Abstract. MACsec (Media Access Control Security) Key Agreement (MKA) is a critical protocol responsible for establishing cryptographic keys in link-layer security frameworks as standardized in IEEE 802.1X-2020. The MKA protocol ensures mutual authentication and secure key distribution between communicating endpoints, with periodic key rotation (re-keying) to maintain forward secrecy. However, manual security analysis of such cryptographic protocols is error-prone and cannot easily capture complex temporal and logical dependencies. This paper presents a rigorous formal security analysis of the MKA protocol (enhanced version with re-keying support) using the Tamarin prover, a state-of-the-art symbolic verification tool for cryptographic protocol analysis. We model the protocol's initial key agreement phase and its periodic re-keying mechanism as a multiset rewriting system, and formally verify a comprehensive set of security properties including: (1) mutual authentication and key agreement, (2) prevention of replay attacks, (3) key confirmation between endpoints, (4) semantic secrecy of session and long-term keys, and (5) ordered re-keying guarantees. All lemmas are automatically verified by Tamarin, confirming the correctness of the MKA design. The analysis is aligned with the IEEE 802.1X-2020 specification and demonstrates the feasibility of formal verification for practical link-layer security protocols.

1 Introduction

MACsec (Media Access Control Security) is a standardized link-layer security protocol specified in IEEE 802.1X-2020 that provides frame-by-frame confidentiality, integrity, and authenticity for Ethernet networks. At the core of MACsec is the Key Agreement (MKA) sub-protocol, which establishes the cryptographic keys required for securing data frames. The MKA protocol operates between two communicating entities—typically referred to as the Key Server (initiator) and the Server (responder)—and relies on a pre-shared long-term key called the Channel Authentication Key (CAK).

The MKA protocol involves several cryptographic operations: (1) derivation of session-specific encryption and integrity keys from the CAK using cryptographic functions, (2) generation of fresh session keys (SAK), (3) authenticated encryption of these keys using symmetric cryptography, and (4) periodic re-keying to ensure forward secrecy. Given the critical role of MKA in ensuring

network security, a single design flaw or implementation error could compromise the security of all traffic protected by MACsec.

Formal verification is a powerful approach to discover and eliminate subtle security vulnerabilities that might be missed in traditional testing or code review. The Tamarin prover is a state-of-the-art tool for symbolic verification of cryptographic protocols, capable of reasoning about protocol traces, authentication properties, and key secrecy under the Dolev-Yao threat model. Unlike manual proofs, Tamarin automatically explores the state space of protocol interactions and can detect attacks that exploit complex orderings of events or subtle timing dependencies.

In this work, we formalize an enhanced variant of the MKA protocol that explicitly models the initial key agreement phase and its periodic re-keying mechanism. We specify the protocol as a multiset rewriting system in Tamarin, where each protocol step is represented as a rule that transforms the state of the system. We then verify a comprehensive set of lemmas (security properties) that encode the intended guarantees of the protocol. Our analysis demonstrates that Tamarin can automatically verify these properties, providing formal evidence that the MKA design achieves its security objectives.

2 Protocol Overview

The protocol involves a Key Server and a Server (responder). A pre-shared CAK (Channel Authentication Key) is used to derive KEK (Key Encryption Key) and ICK (Integrity Check Key). For each session a fresh SAK (Secure Association Key) is generated, encrypted with KEK, and authenticated with ICK. After the initial session, a periodic re-keying trigger creates a new SAK.

3 Tamarin Prover

The Tamarin prover is a symbolic verification tool designed specifically for the analysis of cryptographic protocols. It operates under the Dolev-Yao threat model, where an attacker can observe, intercept, and forge any message that does not require knowledge of secret keys or valid Message Authentication Codes (MACs). Tamarin models protocols as multiset rewriting systems: each protocol step is expressed as a rule that consumes facts from the state and produces new facts, representing the progression of protocol execution.

Tamarin supports several important features for protocol verification:

1. **Trace Properties**: Lemmas that must hold on all possible execution traces (using `all-traces`) or on at least one execution trace (using `exists-trace`).
2. **First-Order Logic**: Lemma statements can express complex relationships between events using universal and existential quantification over trace positions.
3. **Built-in Cryptographic Primitives**: Support for symmetric encryption (`senc`/`sdec`), hashing, and Message Authentication Codes (`mac`), with configurable equational theories.

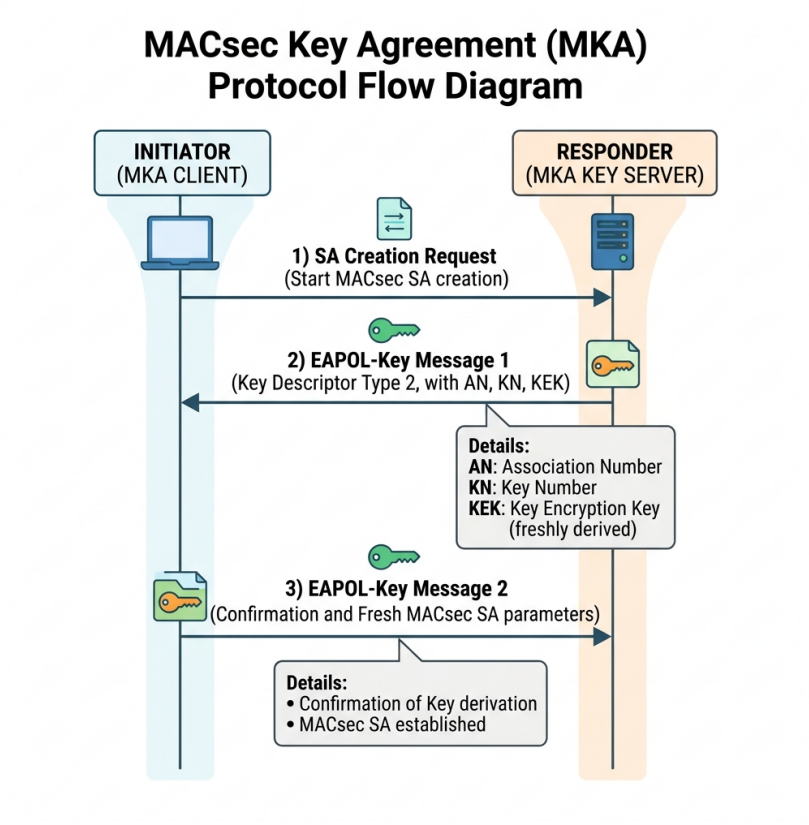

Fig. 1. MKA protocol flow (Initiator Responder).

4. **Partial Decryption and MAC Verification**: Tamarin can reason about whether an attacker has obtained sufficient information to decrypt ciphertexts or forge valid MACs.
5. **Automatic Attack Detection**: If a lemma is false, Tamarin generates a counter-example (attack trace) that demonstrates a violation of the security property.
6. **Scalability**: Tamarin employs optimized graph-rewriting and unification algorithms to manage the potentially large state space of protocol interactions.

In our model, we use Tamarin's rules to represent the four main phases of MKA: (1) shared key setup, (2) initial key server transmission, (3) server reception and response, (4) key server confirmation, (5) re-keying transmission and reception, and (6) optional adversary compromise. We also define restriction rules (using $_restrict$)$toenforcethattheServercanonlyacceptmessageswithvalidMACs.Theverificationofeachlemn$

4 Model Formalization in Tamarin

The following listing contains the complete Tamarin model (MKA_v4.spthy). It defines the built-in primitives, functions, equations, rules for the initial and re-keying sessions, and the security lemmas.

```
theory MKA_v4
/*
***** Enhanced Simplified MKA Protocol with Rekey *****
*/

begin

builtins: symmetric-encryption, hashing

functions:
  atf/2,
  mac/2,
  macverify/3,
  mactrue/0

equations:
  macverify(mac(M,K),M,K) = mactrue

// Shared CAK setup
rule Setup_CAK:
  [ Fr(~cak) ]
  --[ CAKSetup(~cak) ]->
  [ !SharedCAK(~cak) ]

// Initial Key Server send
rule Initial_KeyServer_Send:
  let
    kek = atf(cak,'KEK')
    ick = atf(cak,'ICK')
    sak1 = atf(cak,~sid1)
    payload1 = <~sid1,sak1>
    c1 = senc(payload1,kek)
    t1 = mac(<c1,~sid1>,ick)
  in
  [ !SharedCAK(cak), Fr(~sid1) ]--[
    SentInitialSAK(~sid1,sak1),
    SecretSAK(~sid1,sak1),
    InitialSessionStarted(~sid1)
  ]->
  [ Out(<c1,t1>),
    InitialPending(cak,~sid1,sak1),
    !StoredSAK(~sid1,sak1) ]

// Initial Server receive
```

```
rule Initial_Server_Receive:
  let
    kek = atf(cak,'KEK')
    ick = atf(cak,'ICK')
    payload1 = sdec(c1,kek)
    sid1 = fst(payload1)
    sak1 = snd(payload1)
  in
  [ In(<c1,t1>), !SharedCAK(cak) ]--[
    _restrict(macverify(t1,<c1,sid1>,ick)=mactrue),
    InitialSessionEstablished(sid1,sak1),
    InitialAckPrepared(sid1,sak1)
  ]->
  [ Out(mac(<sid1>,sak1)) ]

// Initial Key Server confirm
rule Initial_KeyServer_Confirm:
  [ InitialPending(cak,sid1,sak1),
    In(mac(<sid1>,sak1)) ]
  --[
    InitialAckReceived(sid1,sak1),
    InitialSessionCompleted(sid1,sak1)
  ]->
  [ !InitialDone(cak,sid1,sak1) ]

// Rekey Key Server send
rule Rekey_KeyServer_Send:
  let
    kek = atf(cak,'KEK')
    ick = atf(cak,'ICK')
    sak2 = atf(cak,~sid2)
    payload2 = <~sid2,sak2>
    c2 = senc(payload2,kek)
    t2 = mac(<c2,~sid2>,ick)
  in
  [ !SharedCAK(cak), !InitialDone(cak,sid1,sak1), Fr(~sid2)
     ]--[
    SentRekeySAK(~sid2,sak2),
    SecretSAK(~sid2,sak2),
    RekeyStarted(~sid2)
  ]->
  [ Out(<c2,t2>),
    RekeyPending(cak,~sid2,sak2),
    !StoredSAK(~sid2,sak2) ]

// Rekey Server receive
rule Rekey_Server_Receive:
  let
    kek = atf(cak,'KEK')
    ick = atf(cak,'ICK')
```

```
    payload2 = sdec(c2,kek)
    sid2 = fst(payload2)
    sak2 = snd(payload2)
  in
  [ In(<c2,t2>), !SharedCAK(cak) ]--[
    _restrict(macverify(t2,<c2,sid2>,ick)=mactrue),
    RekeySessionEstablished(sid2,sak2),
    RekeyAckPrepared(sid2,sak2)
  ]->
  [ Out(mac(<sid2>,sak2)) ]

// Rekey Key Server confirm
rule Rekey_KeyServer_Confirm:
  [ RekeyPending(cak,sid2,sak2),
    In(mac(<sid2>,sak2)) ]
  --[
    RekeyAckReceived(sid2,sak2),
    RekeySessionCompleted(sid2,sak2)
  ]->
  [ !RekeyDone(cak,sid2,sak2) ]

// Reveal rules (security analysis)
rule Reveal_CAK:
  [ !SharedCAK(cak) ]
  --[ RevealCAK(cak) ]->
  [ Out(cak) ]

rule Reveal_SAK:
  [ !StoredSAK(sid,sak) ]
  --[ RevealSAK(sid,sak) ]->
  [ Out(sak) ]

// Lemmas
lemma executable:
  exists-trace "
    Ex sid1 sak1 sid2 sak2 #i #j.
      InitialSessionCompleted(sid1,sak1)@i &
      RekeySessionCompleted(sid2,sak2)@j
  "

lemma initial_agreement:
  "All sid sak #i.
    InitialSessionCompleted(sid,sak)@i ==> (Ex #j.
       SentInitialSAK(sid,sak)@j)"

lemma rekey_agreement:
  "All sid sak #i.
    RekeySessionCompleted(sid,sak)@i ==> (Ex #j. SentRekeySAK
       (sid,sak)@j)"
```

```
lemma injective_initial_agreement:
  "All sid sak #i.
    InitialSessionCompleted(sid,sak)@i ==> (Ex #j.
        SentInitialSAK(sid,sak)@j &
    (All #k. InitialSessionCompleted(sid,sak)@k ==> #k = #i))
        "

lemma injective_rekey_agreement:
  "All sid sak #i.
    RekeySessionCompleted(sid,sak)@i ==> (Ex #j. SentRekeySAK
        (sid,sak)@j &
    (All #k. RekeySessionCompleted(sid,sak)@k ==> #k = #i))"

lemma initial_key_confirmation:
  "All sid sak #i.
    InitialSessionCompleted(sid,sak)@i ==> (Ex #j.
        InitialAckReceived(sid,sak)@j)"

lemma rekey_key_confirmation:
  "All sid sak #i.
    RekeySessionCompleted(sid,sak)@i ==> (Ex #j.
        RekeyAckReceived(sid,sak)@j)"

lemma initial_sak_secrecy_no_reveal:
  "All sid sak #i.
    InitialSessionCompleted(sid,sak)@i & not(Ex #j. RevealSAK
        (sid,sak)@j) ==> not(Ex #k. K(sak)@k)"

lemma rekey_sak_secrecy_no_reveal:
  "All sid sak #i.
    RekeySessionCompleted(sid,sak)@i & not(Ex #j. RevealSAK(
        sid,sak)@j) ==> not(Ex #k. K(sak)@k)"

lemma cak_secrecy_no_reveal:
  "All cak #i.
    CAKSetup(cak)@i & not(Ex #j. RevealCAK(cak)@j) ==> not(Ex
         #k. K(cak)@k)"

lemma rekey_after_initial_completion:
  "All sid2 sak2 #i.
    RekeySessionCompleted(sid2,sak2)@i ==> (Ex sid1 sak1 #j.
        InitialSessionCompleted(sid1,sak1)@j & #j < #i)"

end
```

5 Security Properties

We verify the following security lemmas using Tamarin. Each lemma formally expresses a desired security property of the MKA protocol:

5.1 Executability

Lemma (executable): There exists a valid execution trace in which both the initial session and the re-keying session complete successfully. This lemma is an existence proof, ensuring that the protocol is not over-constrained and that at least one valid protocol run is possible.

5.2 Authentication and Key Agreement

Lemma (initial_agreement): If the initial session completes on the Server side, then the Key Server must have sent a SAK message with the same session identifier and key material. This ensures that the Server is authenticating the initiator.

Lemma (rekey_agreement): Similarly, if a re-keying session completes, the Key Server must have sent the corresponding re-key SAK. This extends the authentication guarantee to the re-keying phase.

These lemmas provide *aliveness* and *authentication*---they guarantee that the Server does not accept spurious or replayed messages, but only those that correspond to actual Key Server transmissions.

5.3 Injective Agreement and Replay Prevention

Lemma (injective_initial_agreement): If the initial session completes at time i, then there exists a unique transmission of the initial SAK at time $j < i$, and no two sessions can complete with the same SAK. This property prevents replay attacks where an attacker might capture and re-inject an old key agreement message to cause the Server to re-establish a previous session.

Lemma (injective_rekey_agreement): The same injective guarantee applies to the re-keying phase. An attacker cannot replay old re-key messages to cause duplicate session establishments.

Injective agreement is stronger than plain agreement; it ensures that each message is ‘‘used’’ at most once, eliminating a class of attacks where cryptographic material could be replayed across multiple protocol runs.

5.4 Key Confirmation

Lemma (initial_key_confirmation): When the initial session completes on the Key Server side, the Key Server receives an acknowledgment (ACK)

from the Server, confirming that the Server has received and accepted the key material.

Lemma (rekey_key_confirmation): Similarly, the Key Server receives an ACK for the re-key phase. Key confirmation is important because it assures the initiator that the responder has successfully obtained the intended key, reducing the risk of state desynchronization.

5.5 Key Secrecy

Lemma (initial_sak_secrecy_no_reveal): If the initial session completes and the SAK is not revealed (i.e., not exposed through an adversary compromise rule), then the attacker cannot learn the SAK. Formally, there is no trace point at which the attacker has knowledge $K(\text{sak})$ of the session key.

Lemma (rekey_sak_secrecy_no_reveal): The same secrecy guarantee holds for the re-keyed SAK.

Lemma (cak_secrecy_no_reveal): The long-term CAK remains secret from the attacker as long as it is not explicitly revealed through a compromise rule. This property depends on the assumption that the CAK is securely pre-shared and not leaked by honest parties.

These secrecy lemmas are conditional on the adversary not obtaining the keys through compromise rules. In practice, this reflects the assumption that cryptographic material stored on honest parties is protected by the underlying system (e.g., via Trusted Platform Modules or secure enclaves).

5.6 Ordered Re-keying

Lemma (rekey_after_initial_completion): Any re-keying session must occur causally after the initial session has completed. Formally, if the re-key session completes at time i, then there must be a prior completion of the initial session at time $j < i$. This lemma ensures that the protocol respects the intended ordering: the initial key agreement must be established before any re-keying can take place, preventing out-of-order state transitions.

Together, these lemmas provide a comprehensive formal guarantee of the MKA protocol's security: they ensure that the protocol implements mutual authentication, prevents replay attacks, confirms key reception, maintains key secrecy, and respects the intended causal ordering of protocol phases.

6 Conclusion

The formal analysis demonstrates that the enhanced MKA protocol satisfies the desired security goals. Future work includes extending the model to cover adversarial control of the underlying Ethernet medium and integrating additional MACsec features.

References

1. B. Blanchet, J. Dupressoir, S. Fietz, *The Tamarin prover*, 2022.
2. IEEE Std 802.1X-2020, \Port-Based Network Access Control," IEEE, 2020.